

[캡스톤 그로쓰 2차 보고서]

고운이(2176017), 한수지(2176400), 손주현(2276160)

팀 21, 투애니원

I: ON, 예비 부모와 0~7세 자녀 부모를 위한

LLM·STT 기반 맞춤형 교육 및 연령별 육아 지원 서비스

목차

1. 팀 정보

1.1 과제명

1.2 팀 정보

1.3 팀 구성원

2. 과제 요약: 초보 부모를 위한 육아 지원 서비스 I:ON

2.1 문제 정의

2.2 기존 연구와의 비교

2.3 제안 내용

2.4 기대 효과 및 의의

2.5 주요 기능 리스트

3. 과제 설계

3.1 요구사항 정의

3.2 전체시스템 구성

3.3 주요 엔진 및 기능 설계

3.4 주요 기능 구현

3.5 기타: 개발현황 정리

3.6 기타: AI 부가 설명

Appendix. 사용된 오픈소스 / 외부 모듈

1. 팀 정보

1.1 과제명: 아이온(I:ON)

예비 부모와 0~7세 자녀 부모를 위한 LLM·STT 기반 맞춤형 교육 및 연령별 육아 지원 서비스

1.2 팀 정보

- 팀 번호: 21
- 팀 이름: 투애니원
- 기본 원칙
 - 목표 지향: 제한된 시간과 자원을 활용하여 최대 성과에 집중한다.
 - 책임감: 역할에 대한 책임감을 갖고 문제 발생 시 소통한다.

1.3 팀 구성원

- 고운이 (팀장, 2176017):
AI 총괄로서 보이스리포트·홈캠 분석 설계·구현·성능검증을 총괄하며, 1차 발표에서 개발 현황 및 그라운드를, 3차발표에서 성능 지표와 개선 성과를 발표한다.
- 한수지 (팀원, 2176400):
백엔드 총괄로서 시스템 아키텍처·DB·API·배포 및 모니터링을 담당하며, 2차/3차 발표에서 구조·통합·운영 성과를 설명한다.
- 손주현 (팀원, 2276160)
프론트엔드 총괄로서 UX 설계·화면 구현·E2E 데모를 책임지고, 1차 발표에서 개발 계획, 2차, 3차 발표에서 시연과 사용자 여정 설명을 맡는다.

2. 과제 요약: 초보 부모를 위한 육아 지원 서비스 I:ON

2.1 문제 정의

본 프로젝트 I:ON(아이:온)은 예비 부모 및 0~7세 자녀를 둔 부모를 대상으로, LLM·STT 기반 맞춤형 교육 및 연령별 육아 지원 서비스를 제공하는 것을 목표로 한다. 부모의 양육 역량을 강화하고 실제 양육 환경에서 도움을 줄 수 있는 실질적인 솔루션을 모바일 앱(Android) 형태로 구현한다.

2.1.1 타겟 사용자

- 예비 부모: 향후 3년 이내 출산 계획이 있는 사람
- 초보 부모: 0-7세 자녀를 둔 양육에 대한 노하우가 부족한 부모

2.1.2 과제 배경

60명의 예비·초보 부모를 대상으로 한 설문조사 및 선행 연구 분석을 통해 확인된 타겟 부모들의 주요 Pain points는 다음과 같다.

- 체계적인 부모 교육 제공처 부족:
대부분 부모가 부모 교육의 필요성을 느끼지만 접근성 한계로 교육 경험이 적다.

- 실전 상황에 적용 가능한 육아 조언 필요:
기존 육아서적·강의는 실제 상황 대처에 활용도가 낮다.
- 실전 상황에 적용 가능한 육아 조언 필요:
바쁜 일상과 비용 부담으로 전문 교육에 참여하기 어렵다.

2.1.3 과제 제안 방향

부모 유형별 (예비 부모, 0-7세 자녀 부모)로 세분화된 타겟팅을 적용하여 타겟 별로 필요한 솔루션을 제공한다. 이때 각 Pain point에 대응하는 모바일 기반 기능을 설계하여 사용자가 언제든지 접근 가능한 양육 지원을 제공한다. 또한 실질적인 도움을 위해 단순 지식 전달을 넘어 실제 행동 개선과 상호작용 지원으로 이어지는 서비스를 구현한다.

2.2 기존 연구와의 비교

현재까지의 모바일 애플리케이션 시장을 살펴보면, 부모 교육을 직접적으로 목적으로 한 앱은 거의 존재하지 않는다. 대부분은 육아 정보 공유 커뮤니티, 아이 성장 기록 앱, 혹은 교육적 콘텐츠를 아동에게 제공하는 형태로 국한된다. 이에 따라 본 프로젝트는 기존 연구 및 서비스와의 차별성을 강조하기 위해, 일반 교육 앱 사례(2.2.1)와 부모 대상 서비스 사례(2.2.2)로 나누어 비교한다.

2.2.1 기존 연구 1: 교육 앱 사례 (Duolingo)

- 비교 목적: 교육 앱이 일반적으로 어떤 형식과 기능을 갖추어야 하는지 확인하고, 이를 부모 교육 앱에 적용할 수 있는 가능성을 탐색한다.
- 특징 및 강점:
 - 게이미фика를 적극 활용하여 학습 지속성을 유도한다.
 - 발음 연습, 롤 플레이, 즉각적 피드백으로 상호작용적 요소가 탑재되어 사용자의 몰입을 강화한다.
 - 짧은 학습 단위와 성취 배지 등으로 동기 부여 체계를 확립한다.
- 부모 교육으로 확장 시 고려 사항
 - 부모 교육은 언어 학습과 달리 민감한 주제(양육 태도, 아동 발달 문제 등)를 다루므로 피드백 방식이 단순 점수화에 머무를 수 없다.
 - 부모의 심리적 저항감을 완화하고 학습 효과를 높이기 위해 공감적인 피드백 구조가 필요하다.
 - 아이와 부모의 실제 상호작용이 핵심이므로 실전 상황에서의 음성 기반 분석과 실전 대비 시뮬레이션을 보강해야 한다.

따라서 Duolingo의 상호작용성과 몰입 유도 요소를 본 과제에 참고하되, 주제의 민감성과 현실 적용성을 고려한 수정이 반드시 필요하다.

2.2.2 기존 연구 2: 부모 대상 서비스 사례

- 비교 목적: 부모를 대상으로 한 앱 또는 부모 교육 관련 서비스의 특징을 살펴보고, 본 과제와의 차별성을 확인한다.
- 사례:
 - **BabyCenter**, 마미톡 앱: 아동 발달 정보, 육아 팁, 부모 커뮤니티 기능을 제공한다.
 - 일부 병원/기관 주도 부모교육 프로그램: 오프라인 강의나 온라인 강좌 중심이며, 이론 전달 위주의 교육이다.
- 특징 및 한계

- 대부분 아이의 발달 정보를 위주로 한 정보 제공과 부모 커뮤니티 지원에 치중하여 실제 부모 행동 개선을 위한 피드백 시스템은 부족하다.
- 교육의 연속성과 개인 맞춤성이 떨어지기에 사용자 개별 상황을 반영하기 어렵다.
- 부모교육은 앱보다는 오프라인 강좌 비율이 높기 때문에 접근성과 지속적인 참여가 낮다.
- 본 과제의 차별성
 - 단순 정보 전달을 넘어 워크북, 보이스 리포트, 챗봇을 통한 상호작용적·맞춤형 피드백을 제공한다.
 - 부모 개인별 상황을 LLM·STT 기반으로 분석하여 실제 행동 변화를 유도한다.
 - 언제 어디서든 접근 가능한 모바일 친화적 구조로 기존 부모 교육의 접근성 문제를 극복한다.

2.3 제안 내용

2.3.1 목표

- 구현 대상: Android 앱 (버전 9 이상 지원)
- 서비스 목표:
 - 예비 부모 및 0-7세 부모의 아이 연령별 pain point 해소
 - 사용자 맞춤형 교육·피드백·상담 기능 제공
 - 지속적으로 활용 가능한 육아 동반자 서비스

2.3.2 필요 기능 (Feature list-up)

2.3.2.1 출산 전·5세 부모를 대상으로 한 이론 교육

- **Pain point:** 체계적인 부모 교육 제공처 부족
- **Solution:** 사용자 정보 및 전문 자료 기반 맞춤형 워크북 콘텐츠 제공
 - **서적명:** 내 아이를 위한 감정 코칭』(존 가트맨), 『부모의 내면이 아이의 세상이 된다』(다니엘 시겔), 『부모 역할 훈련』(토머스 고든) 외 이화여대 아동발달 센터 검수 자료

2.3.2.2 3-7세 부모를 대상으로 한 실제 상황 상호작용 피드백

- **Pain point:** 실전 상황에 활용 가능한 육아 조언 및 피드백 필요
- **Solution:** 보이스 리포트(홈캠 음성 분석)를 통한 양육 태도(상호작용 방식, 말투) 분석 및 개선 피드백 제공

2.3.2.3 전연령 부모를 대상으로 한 상시 질문 창구

- **Pain point:** 구체적이고 즉각적인 조언 부재
- **Solution:** 챗봇을 통한 24시간 맞춤형 양육 상담 및 육아 조언 지원

2.4 기대 효과 및 의의

2.4.1 워크북

- 체계적인 맞춤형 부모 교육 자료 제공을 통한 예비 부모의 이론 기반 역량 강화

- 이화여대 아동발달 센터의 자료 검수를 통한 자료의 신뢰성 확보

2.4.2 보이스 리포트

- 아이와의 실제 상호 작용 분석 및 피드백 제공을 통한 아이의 행동 개선 유도 및 아이와의 관계 증진
- PSDQ 검사(검증된 성향 검사) 기반의 양육 성향 기반의 피드백 제공을 통한 개인화 제공

2.4.3 챗봇

- 24시간 이용 가능한 상담 창구를 통한 지속적인 심리적·실질적 지원
- 이화여대 아동발달 센터의 검수 자료 기반의 상황에 맞는 구체적인 조언 제공 (민감, 부정확한 정보 방지)

2.5 주요 기능 리스트

2.5.1 워크북 기반 교육 기능

- 사용자 정보(부모의 현재 상황, 아이의 발달 단계) 및 OCR 추출한 전문 서적(부모 교육 이론 서적, 서적명 2.3.2.1 참고)을 활용하여 개인 맞춤형 교육 콘텐츠 제공

2.5.2 보이스 리포트 기반 상호작용 피드백

- 아이와 부모 간의 상호작용을 분석하여 구체적인 상황 및 태도 피드백 보고서 반환

2.5.3 챗봇 기반 질의 응답

- LLM, RAG를 활용한 전문 자료 기반의 사용자 맞춤형 조언 및 실시간 답변 제공

2.5.4 기타 부가 기능

- 회원 가입 및 로그인 (개인정보 기반 맞춤형 서비스 제공, 동기 부여를 위한 진행도 저장)
- 부모 유형 테스트 (PSDQ 검사: 부모양육태도·양육방식 검사, Robinson et al., 1995에서 개발), 부모 교육 서비스 시 부모 특성 반영)

- ~~소셜 기능 (부모 간 경험 공유)~~

3. Project-Design (과제 설계)

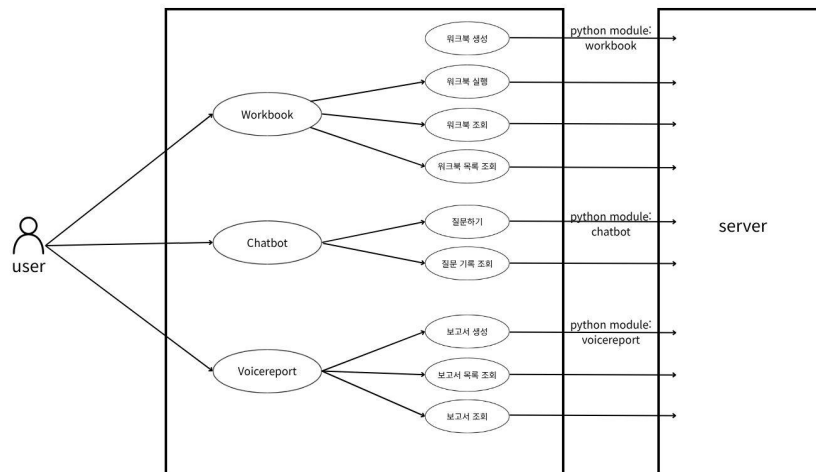
3.1 요구사항 정의

3.1.1 요구사항 정의표

서비스	상세 서비스	비기능 요구사항	기능 설명
1 Workbook	워크북 생성	이해성(초보 부모 특성을 고려한 쉬운 문장 이용)	사용자 정보와 주제를 고려해 맞춤형 워크북 생성 및 저장 (2.3.2.1 전문자료 기반) - 부모 유형별 7가지 단위, 단위 별 최소 4개 액티비티 제공 - 사용자 입력 기반 개인화 요소 최소 3가지(연령, 부모성향, 목표) 반영
	워크북 실행	화면 로딩 2초 이하	맞춤형 워크북 실행, 학습 과정 진행
	워크북 조회		워크북 단건 조회
	워크북 목록 조회		워크북 목록 조회
2 Chatbot	질문하기	질의 응답 평균 응답 시간 ≤ 10초 동시 질의 처리량 ≥ 100 QPS(Query per second)	사용자 정보와 선별된 육아 전문 자료를 바탕으로 육아 조언 제공

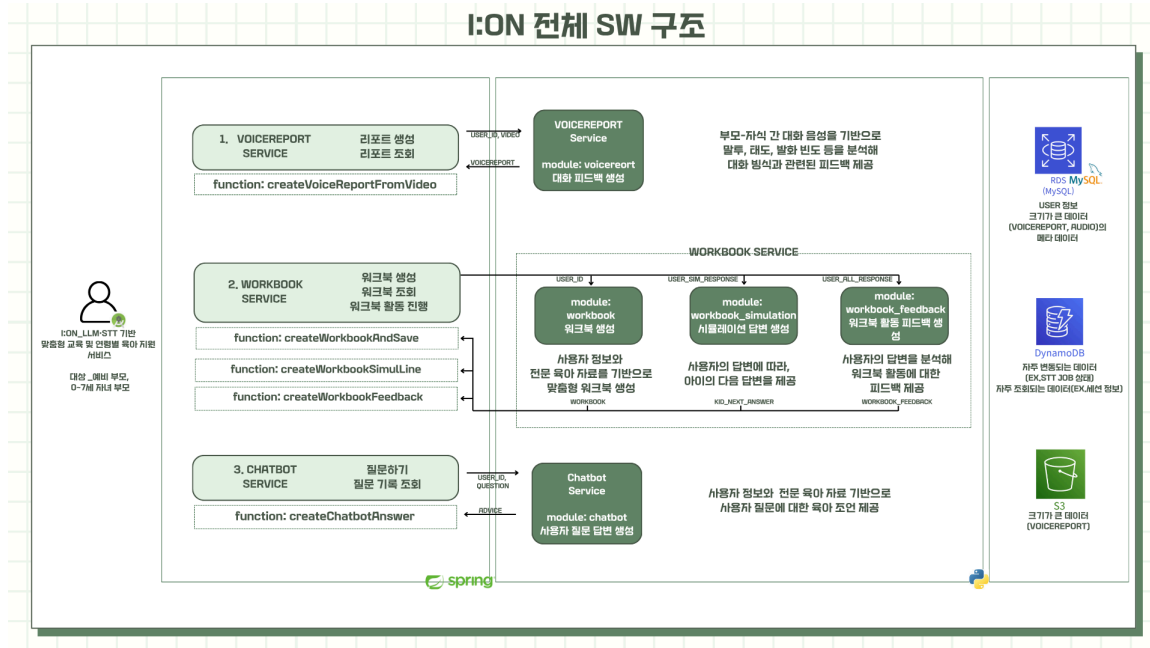
		RAG 검색 결과 Top-k 문서 = 5개	
	질문 기록 조회	사용자 보안성 (채팅 기록 암호화)	이전 Chatbot 이용 기록 조회
3 Voicereport	보고서 생성	STT 전환 정확도 $\geq 85\%$ (사투리 반영) 화자 분리 정확도 $\geq 80\%$ 녹음 파일 입력 허용 용량 $\leq 100\text{MB}$, 길이 $\leq 5\text{분}$	음성 파일 분석을 통해 대화 분석 피드백 리포트 생성 및 저장
	보고서 목록 조회		보고서 목록 조회
	보고서 조회	사용자 보안성 (보고서 암호화, 사용자만 접근 가능)	보고서 단건 조회
4 Community	게시글 쓰기		게시글 업로드
	게시글 조회		게시글 조회

3.1.2 Usecase Diagram



3.2 전체 시스템 구성

3.2.1 시스템 아키텍처 구성도



3.2.1.1 Python Module

(1) workbook

- 워크북 생성: 워크북을 생성하면 spring에서 워크북을 만들어 리턴하는 python 모듈 workbook을 호출, 결과물을 저장
- 워크북 조회: 사용자가 워크북을 조회하면 spring에서 저장된 워크북을 제공
- 워크북 활동 진행: 사용자가 워크북을 실행하면 spring에서 저장된 워크북을 이용해 활동을 진행

(2) chatbot

- 질문하기: 사용자가 질문을 입력해 답변을 요청하면 spring에서 답변을 만들어 리턴하는 python 모듈 chatbot을 호출, 결과물을 저장 및 제공
- 질문 기록 조회: 사용자가 질문 기록을 조회하면 spring에서 저장된 질문, 답변 기록을 제공

(3) voicereport

- 리포트 생성: 사용자가 리포트 생성을 요청하면 spring에서 리포트를 만들어 리턴하는 python 모듈 voicereport를 호출, 결과물을 저장 및 제공
- 리포트 조회: 사용자가 리포트를 조회하면 spring에서 저장된 리포트를 제공

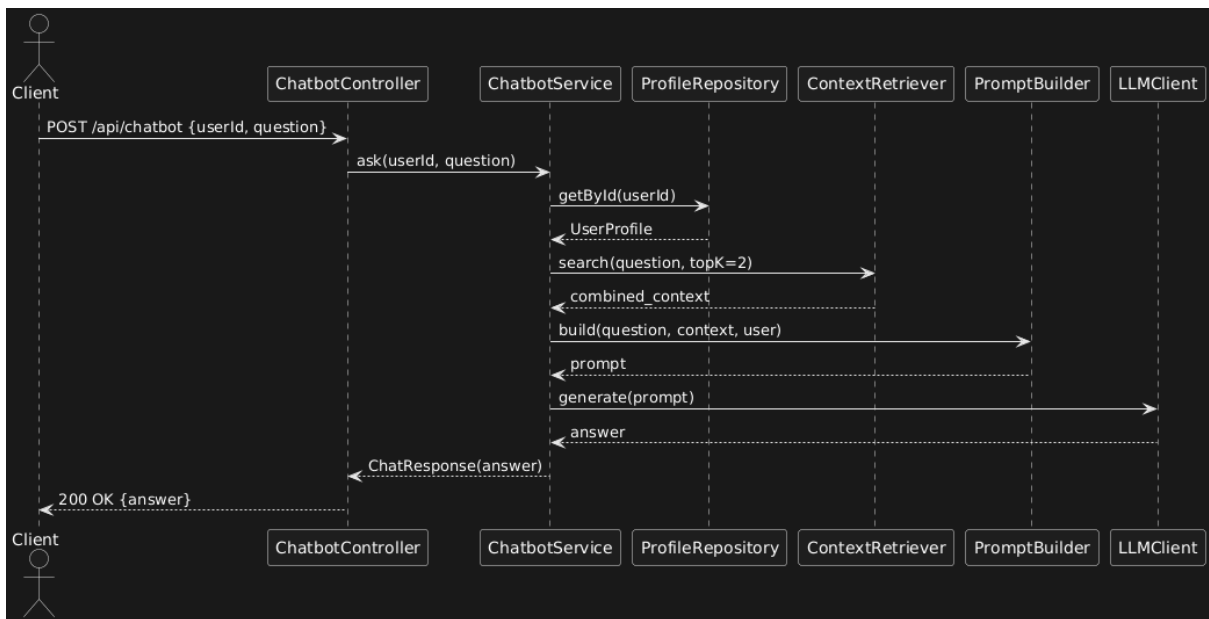
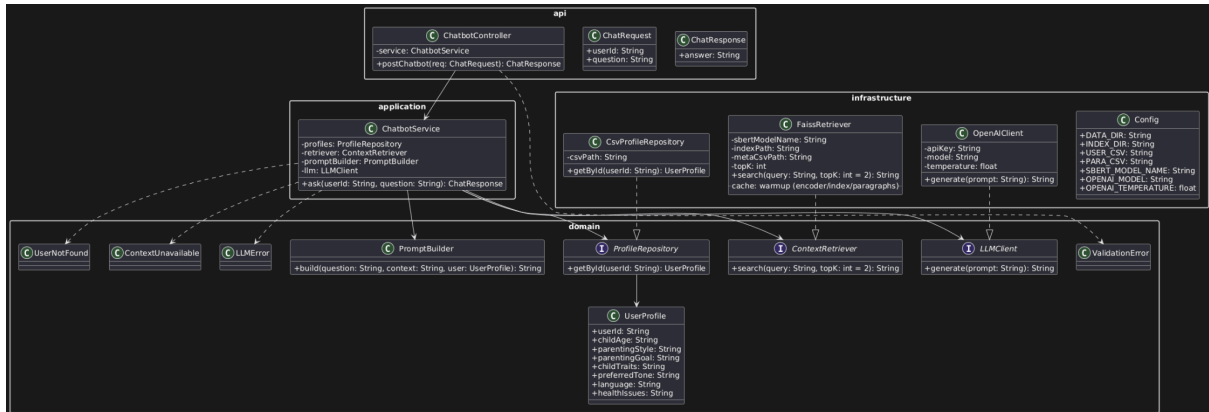
(4) community

~~게시글 쓰기: 사용자가 게시글 업로드를 요청하면 spring에서 결과물을 저장 및 제공~~

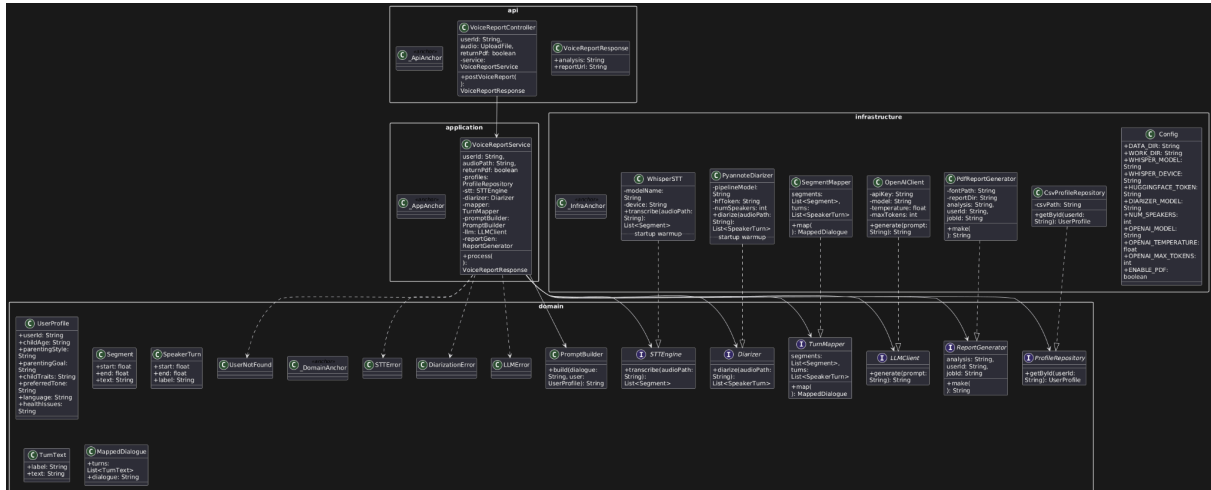
~~게시물 조회: 사용자가 게시글을 조회하면 spring에서 저장된 게시글을 제공~~

3.2.1.2 주요 기능 별 Class Diagram과 Sequence Diagram

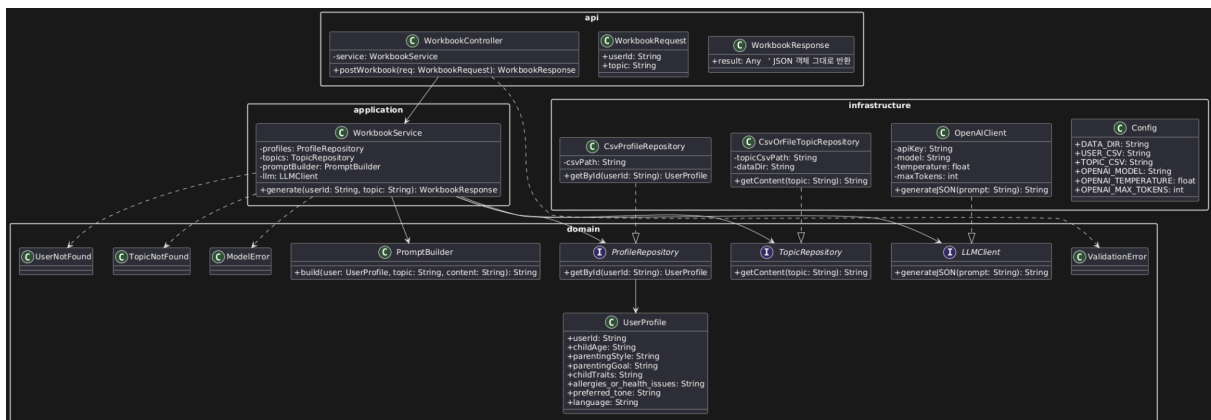
3.2.1.2.1 챗봇 기능

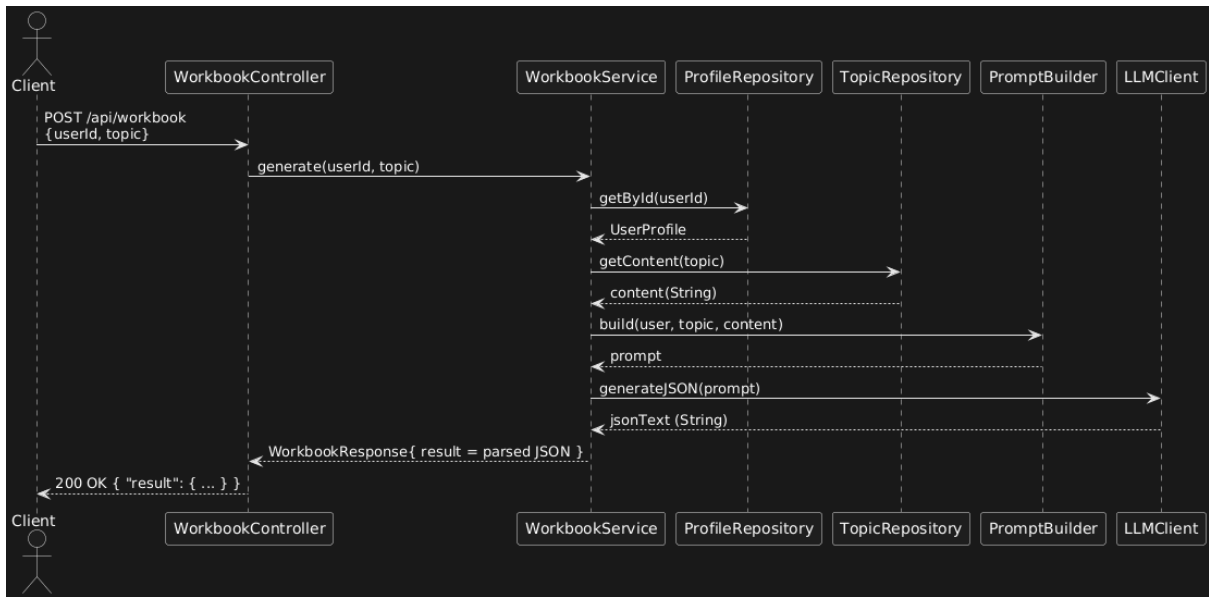


3.2.2.2 보이스리포트 기능



3.2.2.3 워크북 기능





3.3 주요 엔진 및 기능 설계

3.3.1 챗봇 기능

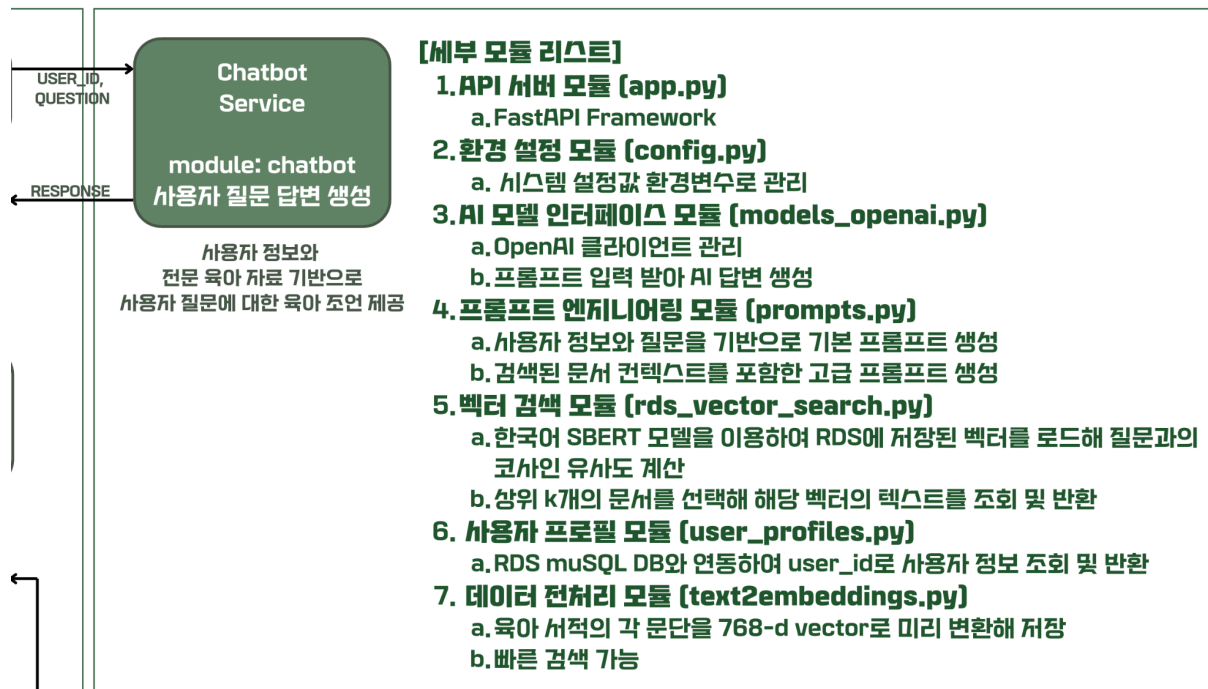
3.3.1.1 시스템 개요

I:ON 챗봇은 RAG(Retrieval-Augmented Generation) 아키텍처를 기반으로 육아 상담을 제공하는 AI 기반 맞춤형 챗봇 시스템입니다. 사용자 프로필 정보와 전문 육아 서적의 내용을 결합하여 개인화된 육아 조언을 제공합니다.

3.3.1.2 핵심 아키텍처

- 웹 프레임워크: FastAPI 0.115.0 기반 RESTful API
- AI 모델: OpenAI GPT-5-mini-2025-08-07
- 임베딩 모델: KR-SBERT-V40K-klueNLI-augSTS (한국어 특화)
- 데이터베이스: AWS RDS MySQL
- 벡터 검색: Sentence Transformers + Cosine Similarity

3.3.1.3 주요 모듈 설계



A. API 서버 모듈 (app.py)

- FastAPI 프레임워크를 사용하여 고성능 비동기 웹 서버를 구축
- Pydantic을 통한 요청/응답 데이터 검증 및 OpenAPI 스키마 자동 생성 기능을 구현
- 커스텀 예외 클래스(ChatbotAPIError)를 정의하여 일관된 에러 응답 포맷을 제공
- 글로벌 예외 핸들러를 통해 모든 에러를 표준화된 JSON 형식으로 반환

B. 환경 설정 모듈 (config.py)

- 시스템 설정값을 환경 변수로부터 로드하여 관리
- OpenAI API 키, 사용할 GPT 모델, 최대 토큰 수 등 핵심 설정을 기본값과 함께 정의하여 안전성 확보

C. AI 모델 인터페이스 모듈 (models_openai.py)

- 싱글톤 패턴을 사용한 OpenAI 클라이언트 관리
- 불필요한 재초기화를 방지, 리소스 효율성 극대화
- Chat Completions API를 통해 프롬프트를 입력받아 AI 답변을 생성

D. 프롬프트 엔지니어링 모듈 (prompts.py)

- 사용자 정보와 질문을 기반으로 기본 프롬프트를 생성
- 검색된 문서 컨텍스트를 포함한 고급 프롬프트 생성 기능을 제공
- 7가지 세부 지침(선택지 제시, 공감적 어투, 의학적 진단 회피, 발달 단계 반영, 단계별 실행 팁, 사용자 정보 맞춤, 참고 자료 활용)을 포함하여 답변 품질 보장

E. 벡터 검색 모듈 (rds_vector_search.py)

- 한국어 SBERT 모델을 싱글톤 패턴으로 관리
- RDS에서 모든 저장된 벡터를 로드하여 질문과의 코사인 유사도를 계산
- 유사도 기준 상위 k개(기본값 2개) 문서를 선택해 해당 텍스트를 조회하여 반환

F. 사용자 프로필 모듈 (user_profiles.py)

- RDS MySQL 데이터베이스와 연동하여 사용자 ID로 프로필 정보를 조회
- 자녀 나이, 양육 스타일, 양육 목표, 아이 성향, 선호 말투, 기본 언어, 건강/특이사항 등 8개 필드를 추출하여 맞춤형 답변 생성에 활용

G. 데이터 전처리 모듈 (data_preprocessing/text2embeddings.py)

- 육아 서적 내용을 문단 단위로 RDS에서 로드
- KR-SBERT 모델을 사용하여 각 문단을 768차원 벡터로 변환
- 배치 처리(batch_size=64)를 통해 효율성을 향상
- 생성된 벡터를 RDS에 저장하여 런타임 검색에 사용

3.3.1.4 데이터베이스 설계

- **user_profile** 테이블: 사용자 프로필 정보 저장
- **emotion_coaching_text** 테이블: 육아 서적 원본 텍스트 저장 (id, paragraph)
- **emotion_coaching_vector** 테이블: 임베딩 벡터 저장 (id, vector BLOB)

3.3.1.5 BACK-END 설계 및 구현

A. ChatbotController: createAnswer (/api/chatbot/{userId})

사용자 질문에 대한 챗봇 대답을 생성하고 저장하는 createChatbotAnswer 함수를 호출합니다. userId, question을 input으로 받고, ChatbotAnswerResponse를 output으로 리턴합니다.

B. ChatbotService: createChatbotAnswer

Python에 사용자 질문에 대한 답변 생성을 요청하는 함수 createAnswer를 호출해 답변을 받고 저장합니다. userId, question을 input으로 받고, ChatbotAnswerResponse를 output으로 리턴합니다.

> Exception: Python 엔진으로 질문 답변 생성을 위임하는 과정에서 입력 검증·외부 API 호출·영속화(저장) 구간을 분리하고, 각 단계별로 구체적인 커스텀 예외(ChatbotException)를 던집니다. 입력 단계에서는 빈 질문/과도한 길이/사용자 미존재 등을 questionEmpty, questionTooLong, userNotFound로 식별합니다. 저장 단계에서는 응답 직렬화/DB 트랜잭션 실패를 persistenceFailure로 처리합니다.

C. PythonAnalysisClient: analyze

사용자 질문, 유저 식별 정보를 Python 분석 서버에 전송하고, WebClient로 응답을 ChatbotAnswerResponse로 받아 처리합니다.

> Exception: 프롭토프 포맷 오류·누락 필드 등을 pythonBadRequest, 모델 과부하/서버 내부 오류·네트워크 장애를 pythonServerError, 응답 지연을 analysisTimeout, 호출 빈도 제한을 rateLimited, 안전성 필터(부적절 콘텐츠 등)에 의한 차단을 policyRefused로 구분합니다.

3.3.2 보이스리포트 기능

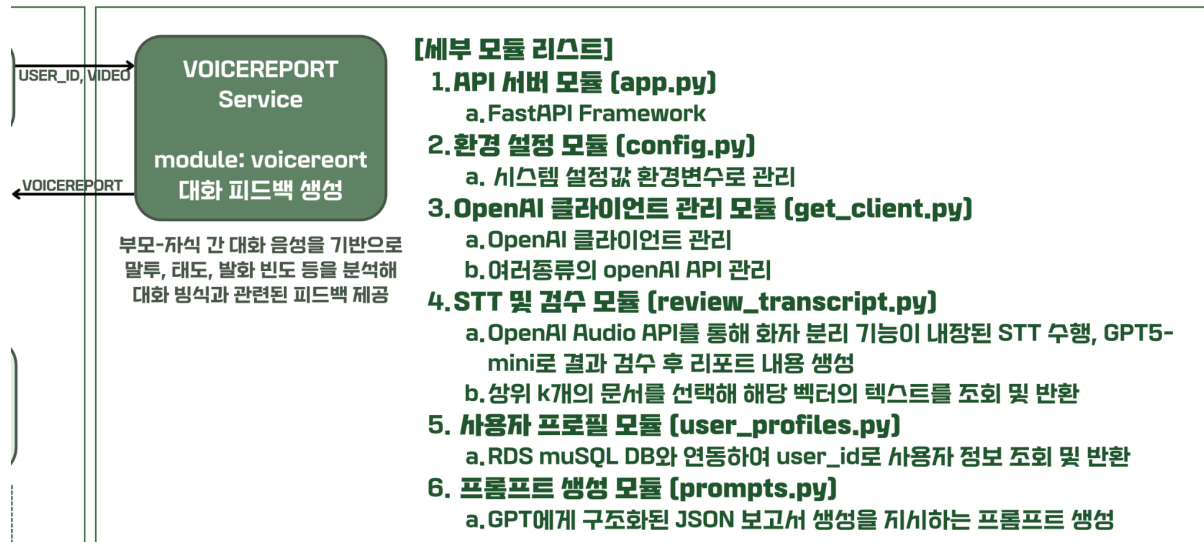
3.3.2.1 시스템 개요

VoiceReport는 부모-자녀 간의 음성 대화를 분석하여 구조화된 양육 피드백 보고서를 생성하는 FastAPI 기반의 RESTful 서비스입니다. 90초 이내의 음성 파일을 업로드하면 화자 분리, 음성 전사, AI 분석을 거쳐 10개 이상의 상세 분석 항목을 포함한 JSON 보고서를 반환합니다.

3.3.2.2 핵심 아키텍처

- 웹 프레임워크: **FastAPI 0.115.0**
- AI 모델: **OpenAI GPT-5-mini-2025-08-07, GPT-4o-transcribe-diarize**
- 데이터베이스: **AWS RDS MySQL (PyMySQL)**
- 음성 처리: **OpenAI Audio API (diarized_json)**
- 검증: **Pydantic 2.9.2**

3.3.2.3 주요 모듈 설계



A. API 서버 모듈 (app.py)

- FastAPI를 사용한 비동기 웹 서버
- 표준화된 에러 핸들링 시스템(VoiceReportAPIError)을 구현
- Pydantic 모델(Expression, ChangeProposalItem, EmotionItem, Emotion, Frequency, VoiceReport)을 통해 응답 데이터를 검증
- Python 표준 라이브러리 wave 모듈로 오디오 길이를 검증하여 90초 초과 시 에러 처리

B. 환경 설정 모듈 (config.py)

- 환경 변수 기반 설정 관리 시스템
- 작업 디렉토리, OpenAI API 설정(모델, Temperature 0.7, Max tokens 10,000) 등을 관리
- pathlib.Path를 사용한 크로스 플랫폼 경로 처리 지원

C. OpenAI 클라이언트 관리 모듈 (get_client.py)

- 안전한 싱글톤 패턴으로 OpenAI 클라이언트를 관리
- threading.Lock을 사용하여 경쟁 조건을 방지
- 범용 GPT 요청 함수(request_gpt)와 특화 래퍼 함수(request_gpt_for_review, request_gpt_for_final) 제공
- OpenAI Audio API 전사 기능(request_transcription)을 통해 화자 분리 기능을 지원

D. 음성 전사 및 검수 모듈 (review_transcript.py)

- OpenAI Audio API를 통해 화자 분리 기능이 내장된 STT를 수행
- GPT-5-mini로 전사 결과를 검수, Parent/Child 재라벨링을 수행
- JSON 추출 유틸리티를 통해 모델 출력에서 안전하게 JSON을 추출
- 정규식 패턴 매칭으로 다양한 API 응답 형식에 대응

E. 사용자 프로필 모듈 (user_profiles.py)

- PyMySQL을 사용한 RDS MySQL 연동
- DictCursor를 통해 결과를 딕셔너리로 반환
- Parameterized Query를 사용하여 SQL Injection을 방지
- 8개 필수 프로필 필드(user_id, child_age, parenting_style, parenting_goal, child_traits, preferred_tone, language, health_issues)를 추출

F. 프롬프트 생성 모듈 (prompts.py)

- GPT에게 구조화된 JSON 보고서 생성을 지시하는 프롬프트를 생성
- JSON 스키마 정의, 역할 정의, 필수 규칙(STT 오류 고려, 프로필 반영, 순수 JSON 출력), 사용자 정보 및 대화 내용 삽입을 포함
- textwrap.dedent와 f-string을 사용하여 토큰 최적화

3.3.2.4 응답 스키마 설계

보고서는 reportId, subTitle, day, conversationSummary, overallFeedback, expression(부모/자녀 표현 분석), changeProposal(표현 개선 제안 2개), emotion(타임라인 및 피드백), kidAttitude, frequency(대화 빈도 분석), strength로 구성됨.

3.3.2.5 BACK-END 설계 및 구현

A. VoiceReportController: createVoiceReport (/api/voice-reports/{userId})

보이스리포트 생성하고 저장하는 createVoiceReportFromVideo 함수를 호출합니다 userId, video, request를 input으로 받고, VoiceReportResponse를 output으로 리턴합니다

```
@PostMapping(
    value = "/{userId}",
    consumes = MediaType.MULTIPART_FORM_DATA_VALUE,
    produces = MediaType.APPLICATION_JSON_VALUE
)
public ResponseEntity<VoiceReportResponse> createVoiceReport(
    @PathVariable("userId") int userId,
    @RequestPart("video") MultipartFile video,
    HttpServletRequest request
) {
    VoiceReportResponse resp = voiceReportService.createVoiceReportFromVideo(userId, video);
    return ResponseEntity.ok(resp);
}
```

codesnap.dev

B. VoiceReportService: createVoiceReportFromVideo

업로드한 비디오의 이용 가능 여부를 확인하고, convertToWav16kMon 함수를 호출해 음성 파일로 변환합니다. 이후, python에 리포트 생성을 요청하는 analyze 함수 호출하고 생성된 리포트를 저장 및 반환합니다. userId, video를 input으로 받고, VoiceReportResponse를 output으로 리턴합니다

> Exception: 업로드 오류, ffmpeg 변환 오류, Python 분석 오류, 그리고 분석 서버 타임아웃 등의 상황에 대해 각각 구체적인 커스텀 예외(VoiceReportException)를 던집니다. 이렇게 구분함으로써 프론트엔드가 사용자에게 어떤 단계에서 문제가 발생했는지 명확히 전달할 수 있고, 서버 로그에서도 원인을 빠르게 파악할 수 있습니다. 코드 작성 시엔 파일 입출력·외부 프로세스(ffmpeg)·외부 API 호출(Python)처럼 실패 가능성이 높은 구간을 분리해 단계별로 안전하게 예외를 처리하도록 설계했습니다.

> Function: convertToWav16kMon: ffmpeg 명령어를 실행해 입력 동영상(input)을 16kHz 모노 WAV(pcm_s16le) 형식으로 변환합니다 변환이 완료되면 출력 파일(outWav)의 존재와 크기를 검증해 정상 변환 여부를 확인합니다. 변환 시간 초과, ffmpeg 실패, 출력 파일 비정상 시 IllegalStateException을 발생시킵니다. 모든 예외 발생 시 임시 출력 파일을 삭제하고 RuntimeException("Audio convert failed: ...")으로 래핑하여 상위로 전달하도록 구현했습니다. ffmpeg 변환 과정에서 발생할 수 있는 다양한 오류(시간 초과·실패·빈 출력)를 세밀히 감지하고, 불완전한 파일이 저장되지 않도록 안전하게 처리하기 위해 다음과

```
private void convertToWav16kMono(Path input, Path outWav) {
    List<String> cmd = List.of(
        ffmpegPath, "-y",
        "-i", input.toAbsolutePath().toString(),
        "-vn", "-ac", "1",
        "-ar", "16000",
        "-acodec", "pcm_s16le",
        "-f", "wav",
        outWav.toAbsolutePath().toString()
    );

    StringBuilder err = new StringBuilder();
    try {
        Process p = new ProcessBuilder(cmd).start();
        try (BufferedReader br = new BufferedReader(new InputStreamReader(p.getErrorStream()))) {
            String line;
            while ((line = br.readLine()) != null) err.append(line).append(System.lineSeparator());
        }
        boolean ok = p.waitFor(convertTimeoutSec, TimeUnit.SECONDS);
        if (!ok) { p.destroyForcibly(); throw new IllegalStateException("ffmpeg timeout (" + convertTimeoutSec + "s)"); }
        if (p.exitValue() != 0) throw new IllegalStateException("ffmpeg failed (exit=" + p.exitValue() + ")\\n" + err);
        if (!Files.exists(outWav) || Files.size(outWav) == 0) throw new IllegalStateException("ffmpeg produced empty output");
        log.info("[VOICEREPORT] ffmpeg OK -> {}", outWav);
    } catch (Exception e) {
        FileUtils.deleteQuietly(outWav.toFile());
        throw new RuntimeException("Audio convert failed: " + e.getMessage() + "\\n" + err, e);
    }
}
```

codesnap.dev

같이 구현했습니다

C. PythonAnalysisClient: analyze

음성 데이터를 Multipart 형식으로 Python 분석 서버에 전송하고, WebClient로 응답을 VoiceReportResponse로 받아 처리합니다.

> Exception: 책임 구간을 명확히 구분해 문제 원인을 신속히 파악하고 대응하기 위해 크게 클라이언트 입력 오류(PythonBadRequestException)와 서버 내부 오류(PythonServerException)로 나누어 예외 처리를 진행했습니다. 클라이언트 측 오류로는 업로드된 파일이 없거나(videoEmpty), 지원하지 않는 형식이거나(videoUnsupported), 손상되었거나(videoCorrupted), 길이 제한을 초과한(audioTooLong) 경우 예외 처리를 진행합니다. 서버 측에서는 요청한 보이스 리포트가 존재하지 않거나(notFound), Python 분석 과정에서 응답 지연 또는 실패로 인해(analysisTimeout) 예외가 발생할 경우 처리를 진행합니다.

3.4 주요 기능의 구현

3.4.1 챗봇 기능

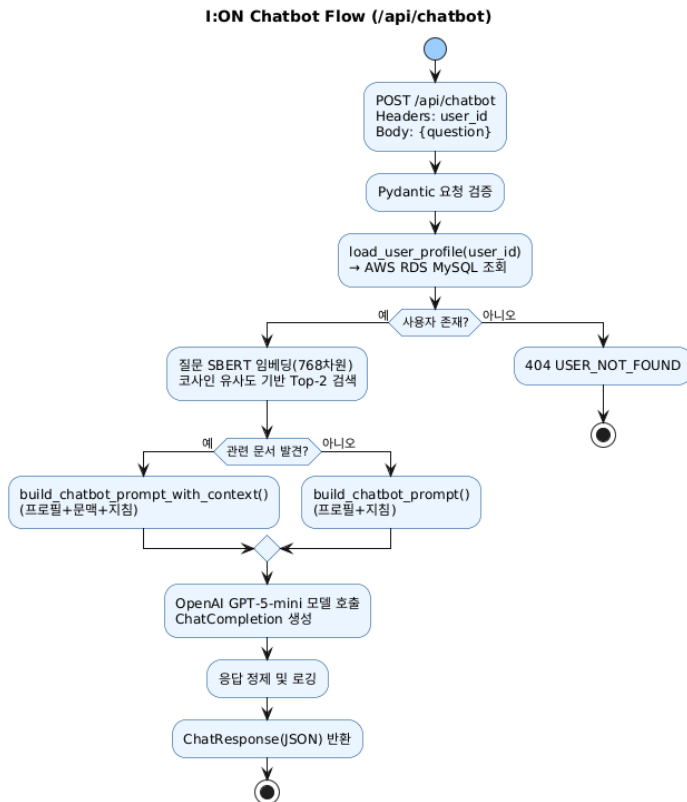
3.4.1.1 시스템 초기화 프로세스

- 서버 시작 시 FastAPI 애플리케이션 인스턴스를 생성
- .env 파일에서 환경 변수를 로드합니다. uvicorn 통합 로깅을 설정하고 예외 핸들러를 등록하며, 필요한 모듈(user_profiles, prompts, models_openai, rds_vector_search)을 임포트합니다. GET /(헬스 체크), POST /api/chatbot(챗봇 대화) 엔드포인트를 노출합니다.

3.4.1.2 사전 데이터 준비 단계

육아 관련 전문 서적 내용을 문단 단위로 분할하여 RDS emotion_coaching_text 테이블에 저장합니다. text2embeddings.py를 실행하여 모든 문단을 조회하고, KR-SBERT 모델로 각 문단을 768차원 벡터로 변환합니다. 배치 처리(batch_size=64)로 효율성을 향상시키고, 생성된 벡터를 emotion_coaching_vector 테이블에 BLOB 타입으로 저장합니다.

3.4.1.3 런타임 요청 처리 FLOW



Phase 1: 요청 수신 및 검증 클라이언트의 POST 요청을 수신하고, Body(question 필드)와 Header(user_id)를 파싱합니다. Pydantic을 통해 필드 존재 여부를 검증하며, 검증 실패 시 422 에러를 자동 반환합니다.

Phase 2: 사용자 컨텍스트 수집 load_user_profile(user_id)을 호출하여 RDS에서 사용자 프로필을 조회합니다. 사용자가 존재하지 않으면 KeyError를 발생시켜 404 에러를 반환합니다. 조회 성공 시 8개 필드(user_id, child_age, parenting_style, parenting_goal, child_traits, preferred_tone, language, health_issues)를 포함한 딕셔너리를 반환합니다.

Phase 3: 관련 문서 검색 (RAG) KR-SBERT 모델 인스턴스를 싱글톤으로 획득하고, 질문 텍스트를 768차원 벡터로 변환합니다. RDS에서 모든 저장된 벡터를 로드하고(SELECT id, vector FROM emotion_coaching_vector), 질문 벡터와 각 문서 벡터의 코사인 유사도를 계산합니다. 정규화(L2 norm으로 단위 벡터 변환)를 수행하고, 유사도를 내적으로 계산하여(-1~1 범위) 상위 k개(기본값 2개) 문서를 선택합니다. 선택된 ID로 실제 텍스트를 조회(SELECT id, paragraph FROM emotion_coaching_text WHERE id IN (...))하고, 검색 실패 시 빈 컨텍스트로 진행(fallback)합니다.

Phase 4: 프롬프트 생성 검색 결과가 있으면 build_chatbot_prompt_with_context()를 호출하여 사용자 정보, 질문, 참고 자료(검색된 문서), 답변 지침(7가지 세부 규칙)을 포함한 프롬프트를 생성합니다. 검색 결과가 없으면 build_chatbot_prompt()를 호출하여 기본 프롬프트를 생성합니다.

Phase 5: AI 답변 생성 싱글톤 패턴으로 OpenAI 클라이언트를 획득하고, Chat Completions API를 호출합니다. system 메시지("당신은 육아 상담 챗봇입니다.")와 user 메시지(생성된 프롬프트)를 전달하며, GPT-5-mini 모델을 사용합니다. API 응답에서 choices[0].message.content를 추출하고 공백을 제거합니다.

Phase 6: 응답 및 로깅 생성된 프롬프트를 서버 로그에 기록하고, ChatResponse 모델로 답변을 반환합니다. HTTP 200 OK 상태로 application/json 형식의 응답을 클라이언트에 전송합니다.

3.4.1.4 에러 처리 메커니즘

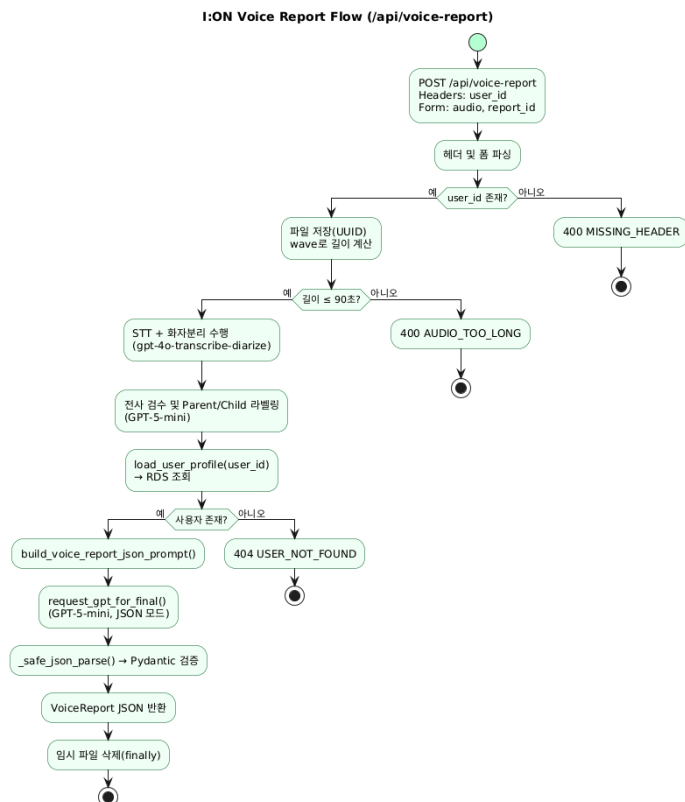
KeyError 발생 시(사용자 미존재) USER_NOT_FOUND 에러(404)를 반환하고, ChatbotAPIError는 재발생시킵니다. 기타 예외는 INTERNAL_ERROR(500)로 처리하며, 백터 검색 실패 시 예외를 catch하고 빈 컨텍스트로 진행하여 서비스 연속성을 보장합니다.

3.4.2 보이스리포트 기능

3.4.2.1 시스템 초기화 프로세스

서버 시작 시 FastAPI 애플리케이션 인스턴스를 생성하고, @app.on_event("startup")으로 OpenAI 클라이언트를 사전 초기화합니다. GET /(기본 정보), GET /healthz(헬스체크) 엔드포인트를 노출하며, 환경 변수를 로드하고 로깅을 설정합니다.

3.4.2.2 런타임 요청 처리 FLOW



Phase 1: 요청 수신 및 검증 클라이언트의 POST /api/voice-report 요청을 수신하고, Headers(user_id), Form Data(audio 파일, report_id)를 파싱합니다. user_id 헤더가 없으면 MISSING_HEADER 에러(400)를 발생시킵니다.

Phase 2: 파일 저장 및 길이 검증 업로드된 오디오 파일을 UUID 기반 고유 파일명으로 {WORK_DIR}/uploads/에 저장합니다. Python 표준 라이브러리 wave 모듈로 오디오 파일을 열고, 프레임 수와 샘플링 레이트로 재생 시간을 계산(duration = frames / frame_rate)합니다. 90초를 초과하면 AUDIO_TOO_LONG 에러(400)를 발생시킵니다.

Phase 3: 음성 전사 및 화자 분리 process_audio_for_review()를 호출하여 STT 처리를 수행합니다. 싱글톤 클라이언트를 획득하고 OpenAI Audio API를 호출(model: gpt-4o-transcribe-diarize, response_format: diarized_json, chunking_strategy: auto)하여 화자 정보가 포함된 전사 결과를 얻습니다. 전사 결과를 Dict로 변환하고, GPT-5-mini 검수 모델을 호출하여 Parent/Child 라벨링을 수행합니다. 검수 프롬프트는 역할 정의, 입력 데이터, 출력 규칙(Parent/Child 필드, 타임스탬프 보존), 화자 구분 로직(문맥, 발화 길이, 친밀도 분석)을 포함합니다. 검수된 segments를 "Parent: text\nChild: text" 형식의 대화 문자열로 변환하여 반환합니다.

Phase 4: 사용자 프로필 로드 `load_user_profile(user_id)`를 호출하여 RDS MySQL에서 사용자 프로필을 조회합니다. Parameterized Query(`SELECT * FROM user_profile WHERE user_id = %s`)를 사용하여 SQL Injection을 방지하고, 8개 필드를 추출합니다. 사용자가 존재하지 않으면 `KeyError`를 발생시켜 `USER_NOT_FOUND` 에러(404)를 반환합니다.

Phase 5: 프롬프트 생성 `build_voice_report_json_prompt(dialogue, user_info)`를 호출하여 GPT 프롬프트를 생성합니다. JSON 스키마(`reportId, subTitle, day, conversationSummary, overallFeedback, expression, changeProposal, emotion, kidAttitude, frequency, strength`)를 정의하고, 역할 정의("부모-아이 대화 코칭 보고서 생성 비서"), 필수 규칙(STT 오류 고려, 프로필 반영, 순수 JSON 출력, 한국어 자연스러운 작성)을 포함합니다. 사용자 정보(`child_age, parenting_style, parenting_goal, child_traits, preferred_tone, language, health_issues`)와 대화 내용(Parent/Child 라벨링된 대화문)을 삽입합니다. `dedent()`와 `strip()`으로 불필요한 공백을 제거하여 토큰을 절약합니다.

Phase 6: 최종 보고서 생성 `request_gpt_for_final()`을 호출하여 GPT-5-mini에 프롬프트를 전달하고 JSON 텍스트를 생성합니다. 시스템 메시지("최종 출력은 반드시 유효한 JSON만 출력하라")를 포함하여 순수 JSON만 반환하도록 지시합니다.

Phase 7: JSON 파싱 및 검증 `safe_json_parse()`를 호출하여 GPT 출력에서 JSON을 추출합니다. 마크다운 코드 블록(``json``)을 제거하고 표준 `json.loads()`를 시도합니다. 실패 시 정규식으로 `{...}` 또는 `[...]` 패턴을 탐색하여 파싱합니다. `reportId` 필드를 설정(또는 정규화)하고, Pydantic 모델(`VoiceReport`)로 검증합니다.

Phase 8: 응답 반환 및 정리 검증된 보고서를 JSON 형식으로 반환(HTTP 200 OK)합니다. `finally` 블록에서 임시 오디오 파일을 삭제(`missing_ok=True`)하여 디스크 공간을 확보합니다.

3.4.2.3 에러 처리 메커니즘

`VoiceReportAPIError`는 구조화된 JSON 응답(`success: false, error: {code, message, details}`)으로 변환됩니다. `HTTPException(422)`은 FastAPI 기본 처리를 사용하고, `KeyError(user_id)`는 `USER_NOT_FOUND(404)`로, `Duration > 90s`는 `AUDIO_TOO_LONG(400)`로, 기타 예외는 `INTERNAL_ERROR(500)`로 처리됩니다. 전역 예외 핸들러(`@app.exception_handler`)로 모든 에러를 표준화합니다.

3.5 기타: 개발 현황 정리

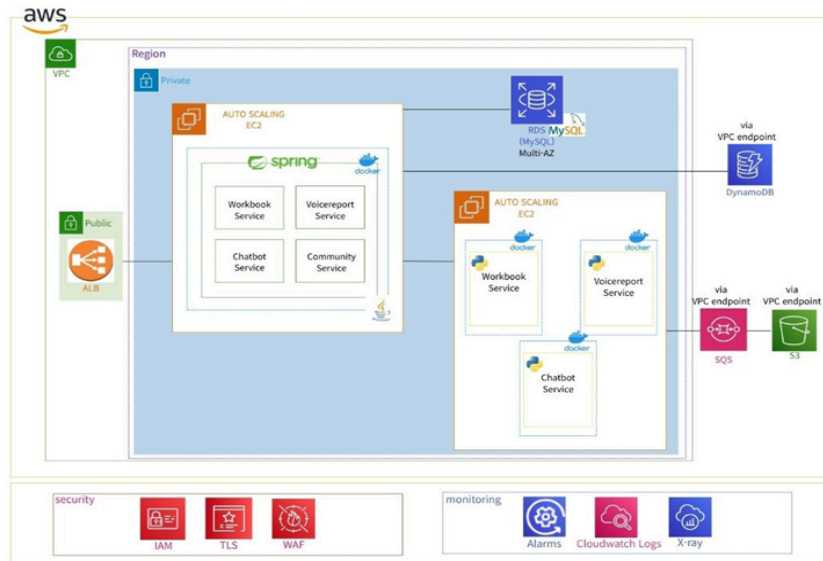
3.5.1 BACK-END

3.5.1.1 데이터 저장 설계: AWS의 세 가지 서비스를 조합하여 처리

1. RDS: 사용자 정보, 게시글 등 구조적인 데이터 저장
2. DynamoDB: 챗봇 대화 기록, STT 분석 결과 등 빈번히 조회·변경되는 데이터 관리
3. S3: 육아 관련 문서, 워크북, 사용자 업로드 음성 파일 등 대용량 비정형 데이터 저장

3.5.1.2 서버 인프라 구축: 기능에 따라 두 개의 EC2 인스턴스로 분리

1. 애플리케이션 서버: Spring Boot 기반 비즈니스 로직, API 요청 처리 담당
2. 음성인식 처리 서버: STT 및 Python 코드 실행



- 개발 완료 사항: 아키텍처 설계, 지도교수님 피드백
- 추가 개발 사항: 핵심 기능(voiceport, chatbot, workbook) 코드 정제, 서버 구축

3.5.1.3 Spring

1. **workbook**: 워크북 생성, 조회, 활동 등 핵심 api 구현 완료
 2. **chatbot**: 질문하기, 기록 조회 등 핵심 api 구현 중
 3. **voicereport**: 리포트 생성, 조회 등 핵심 api 구현 완료
- 추가 개발/ 변경 사항
 - (1) 사용자 관련 서비스(회원가입 및 로그인), 커뮤니티, 홈 기능의 API를 구현
 - (2) 핵심 기능(voiceport, chatbot, workbook)의 코드를 정제하여 효율적으로 개선할 예정

3.5.2 AI

- AI 기반 핵심 세 가지(챗봇, 보이스리포트, 워크북) 기능의 end-to-end flow를 검증 완료. (스타트)

- 스타트 단계에서 여러 파이썬 스크립트로 독립적으로 구현된 각 AI 기반 핵심 기능을 FastAPI기반의 REST API 형태로 통합. (그로쓰)

3.5.2.1 챗봇 기능의 검증된 Flow

[사용자 질문 입력 및 정보 조회] → [SBERT 임베딩 및 FAISS 검색] → [GPT prompt 구성] → [응답 리턴]

- 추가 개발/ 변경 사항
 - (1) API 엔드포인트(POST /chatbot)로 기능을 묶어, userId와 question을 입력 받도록 함.
 - (2) 사용자 질의의 빠른 검색을 위해 SBERT 임베딩과 FAISS 인덱스를 사전에 구축하여, 매번 전체 데이터를 불러오지 않고 즉시 유사 문단 검색이 가능하도록 개선.

3.5.2.2 보이스리포트 기능의 검증된 Flow

[사용자 정보 로드 및 오디오 파일 업로드] → [STT 및 화자 분리] → [Whisper/ Pyannote 결과 매핑] → [prompt 구성] → [피드백 리턴]

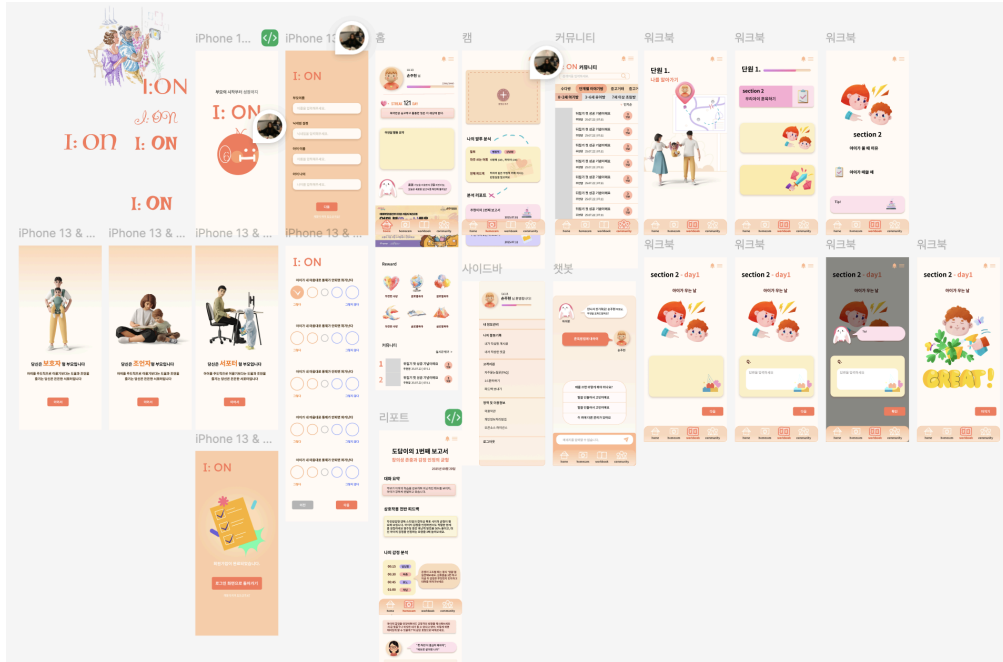
- 추가 개발/ 변경 사항
 - (1) API 엔드포인트(POST /voiceReport)로 묶고, userId와 음성 파일을 입력으로 받도록 구현.
 - (2) STT(whisper)와 화자 분리(Pyannote.audio) 모두 API 내부에서 자동 호출하도록 통합.

3.5.2.3 워크북 기능의 검증된 Flow

[사용자 정보 로드] → [참고 육아 문서 로드] → [프롬프트 생성] → [워크북 구성 내용 리턴]

- 추가 개발/ 변경 사항
- (1) API 엔드포인트(POST /workbook)로 통합.
- (2) userId와 topic을 받아, 내부적으로 해당 토픽 콘텐츠를 조회 후 프롬프트에 통합하도록 수정

3.5.3 FRONT-END



- 와이어 프레임 설계 (100% 완료): 주요 기능을 포함한 모든 화면에 대한 UI 설계
- 주요 기능에 대한 화면 구현 (90% 완료): 홈, 워크북, 보이스리포트, 챗봇 일부
- 주요 기능 간 화면 이동 구현 (90% 완료): 네비게이션 바, 버튼 로직 구현, 구현 안된 기능 제외 완성
- 주요 기능에 대한 서버 연동 및 테스트/end-to-end flow 검증(60%): 워크북, 보이스리포트 완성
- 세부 기능 구현(20%): 챗봇 세부 요소, 보상 체계 및 회원가입 구현 필요

3.5.3.3 AI

스타트 단계에서 여러 파이썬 스크립트로 독립적으로 구현된 각 AI 기반 핵심 기능을 FastAPI기반의 REST API 형태로 통합.

1) 챗봇 기능

- 추가 개발/ 변경 사항
- (3) API 엔드포인트(POST /chatbot)로 기능을 묶어, userId와 question을 입력 받도록 함.
- (4) 사용자 질의의 빠른 검색을 위해 SBERT 임베딩과 FAISS 인덱스를 사전에 구축하여, 매번 전체 데이터를 불러오지 않고 즉시 유사 문단 검색이 가능하도록 개선.

2) 보이스리포트 기능

- 추가 개발/ 변경 사항
- (3) API 엔드포인트(POST /voiceReport)로 묶고, userId와 음성 파일을 입력으로 받도록 구현.
- (4) STT(whisper)와 화자 분리(Pyannote.audio) 모두 API 내부에서 자동 호출하도록 통합.

3) 워크북 기능

- 추가 개발/ 변경 사항
- (3) API 엔드포인트(POST /workbook)로 통합.
- (4) userId와 topic을 받아, 내부적으로 해당 토픽 콘텐츠를 조회 후 프롬프트에 통합하도록 수정

3.6 기타: AI 부가 설명

3.6.1 챗봇 기능의 AI 기술

3.6.1.1 RAG (Retrieval-Augmented Generation) 아키텍처

- RAG는 검색 기반 증강 생성 기술로, LLM의 환각(hallucination)을 방지하고 전문 지식을 주입하기 위해 사용됩니다. 본 시스템은 Retrieval(관련 문서 검색), Augmentation(검색 결과를 프롬프트에 통합), Generation(LLM으로 답변 생성)의 3단계로 구성됩니다.
- 지식 베이스는 전문 육아 서적을 데이터 소스로 구축되었으며, 원본 텍스트는 emotion_coaching_text 테이블에, 임베딩 벡터는 emotion_coaching_vector 테이블에 저장되어 ID를 통해 1:1 매핑됩니다.
- 임베딩 모델로는 KR-SBERT-V40K-klueNLI-augSTS를 선택했습니다. 이 모델은 한국어에 특화되어 있으며, KLUE NLI 데이터셋으로 학습되어 한국어 자연어 추론 성능이 우수합니다. STS(Semantic Textual Similarity) 성능도 뛰어나며, 768차원 dense vector를 생성하여 의미적으로 유사한 문장을 벡터 공간에서 가까이 배치합니다.
- 검색 메커니즘은 다음과 같이 동작합니다.
 - (1) 먼저 사용자 질문을 SBERT로 인코딩하여 동일한 벡터 공간(768차원)으로 매핑합니다.
 - (2) 그 다음 코사인 유사도를 측정합니다. 코사인 유사도는 $\text{similarity}(A, B) = (A \cdot B) / (\|A\| \times \|B\|) = \cos(\theta)$ 로 계산되며, -1에서 1 사이의 값을 가지고 1에 가까울수록 유사합니다. 이 메트릭은 벡터 크기에 불변하며, 의미적 유사성 측정에 효과적이고, 내적 연산으로 계산 효율성이 높습니다.
 - (3) 마지막으로 유사도 기준 상위 k개 문서를 선택합니다. k=2를 선택한 이유는 프롬프트 길이 제한을 고려하고, 관련성 높은 문서에 집중하며, 응답 속도와 품질의 균형을 맞추기 위해서입니다.
- 컨텍스트 주입은 검색된 문서를 프롬프트의 "[참고 자료]" 섹션으로 추가하여 수행됩니다. 명확한 구분자("-- 문서 1 --", "-- 문서 2 --")를 사용하고, 관련도 높은 순서를 보존합니다. AI에게 참고 자료 활용 방법을 명시하고, 전문 자료 기반 답변임을 밝히도록 지시합니다.

3.6.1.2 LLM (Large Language Model) 활용

- **OpenAI GPT-5-mini-2025-08-07** 모델을 선택한 이유는 최신 GPT-5 아키텍처의 성능, mini 버전으로 비용 최적화, 빠른 응답 시간, temperature 자동 최적화 기능, 최대 10,000 토큰 지원 때문입니다.
- 프롬프트 엔지니어링 전략은 다음과 같습니다.
 - System Message로 역할을 설정하여({"role": "system", "content": "당신은 육아 상담 챗봇입니다."}) AI의 페르소나를 정의하고 일관된 어조 및 전문성을 유지합니다.
 - 구조화된 프롬프트는 사용자 정보 섹션(7가지 프로필 정보), 참고 자료 섹션(검색된 전문 서적 내용), 답변 지침 섹션(7가지 세부 규칙: 강요하지 않고 선택지 제시, 공감적 어투 유지, 의학 진단 회피, 발달 단계 반영, 단계별 실행 팁, 사용자 정보 맞춤, 참고 자료 활용)으로 구성됩니다.
 - Few-Shot 프롬프팅을 통해 지침에 구체적인 예시를 제공하여 AI가 원하는 형식으로 답변을 생성하도록 유도합니다. 분량 제어("8~14문장 내로 간결하게 작성")를 통해 사용자 가독성을 향상시키고, 토큰 사용량을 절감하며, 핵심 정보에 집중합니다.
- 개인화 AI 시스템은 사용자 프로필 기반 맞춤화를 통해 구현됩니다.
 - : 자녀 나이(발달 단계에 맞는 조언), 양육 스타일(부모의 가치관 반영), 양육 목표(목표 지향적 조언), 아이 성향(개별 특성 고려), 선호 말투(의사소통 스타일 조정), 언어(다국어 지원 가능), 건강/특이사항(안전성 고려)을 프롬프트에 명시하여 AI가 컨텍스트로 활용하도록 합니다. 맥락

인식 답변은 RAG를 통해 질문과 관련된 전문 지식을 자동 검색하고, 사용자가 명시하지 않은 배경 지식을 보완합니다. System message로 챗봇 역할을 고정하고 프롬프트 구조를 표준화하여 일관성을 유지합니다.

3.6.1.3 안전성 및 윤리 고려사항

의학적 조언 제한을 위해 "의학/진단 등 전문 판단이 필요한 내용은 피하고, 필요한 경우 전문가 상담을 권유하세요"라는 지침을 프롬프트에 명시적으로 포함하여 법적 책임 및 안전 문제를 방지합니다. 강요 방지를 위해 "강요하지 않고 선택지를 제시하세요"라는 지침을 통해 부모의 자율성을 존중하고 윤리적 AI를 구현합니다. 공감적 어조를 유지하기 위해 "공감적이고 따뜻한 어투를 유지하세요"라는 지침으로 사용자 경험을 개선하고 정서적 지지를 제공합니다.

3.6.1.4 AI 성능 최적화

- 레이턴시 최적화를 위해 SBERT 모델 싱글톤 패턴을 사용하여 모델을 재사용하고 로딩 시간을 절약(약 2~3초)합니다. Top-K를 2로 제한하여 검색 범위를 제한하고, 프롬프트 길이를 최소화하며, OpenAI API 응답 시간을 단축합니다. FastAPI의 비동기 기능을 활용하여 향후 개선이 가능합니다.
- 비용 최적화를 위해 GPT-5-mini를 선택하여 GPT-4 대비 저렴한 비용으로 품질과 비용의 균형을 맞췄습니다. 답변 길이를 8~14문장으로 제한하고 불필요한 컨텍스트를 최소화하여 토큰 사용량을 제어합니다. 향후 자주 묻는 질문(FAQ) 캐싱과 중복 API 호출 방지를 통해 추가 최적화가 가능합니다.

3.6.1.5 품질 보장 메커니즘

프롬프트 검증을 통해 명확한 섹션 구분, 7가지 세부 규칙 명시, Few-shot learning을 위한 예시 제공으로 답변 품질을 보장합니다. 출처 명시를 위해 "전문 자료로부터 기반한 답변임을 가볍게 언급하세요"라는 지침으로 신뢰성을 향상시키고 사용자에게 확신을 제공합니다. Fallback 메커니즘을 구현하여 RAG 실패 시에도 기본 프롬프트로 전환하여 서비스 연속성을 보장합니다.

3.6.2 보이스리포트 기능의 AI 기술

3.6.2.1 음성 인식 및 화자 분리 기술

- OpenAI Audio API의 gpt-4o-transcribe-diarize 모델을 사용하여 음성 전사와 화자 분리를 동시에 수행합니다. 이 모델은 90초 이내의 오디오를 처리할 수 있으며, diarized_json 형식으로 각 발화에 화자 정보(SPEAKER_00, SPEAKER_01)와 타임스탬프(start, end)를 포함한 결과를 반환합니다. chunking_strategy를 auto로 설정하여 긴 오디오를 자동으로 분할 처리하며, 한국어 음성 인식에 최적화되어 있습니다.
- 화자 분리 정확도는 음향적 특징(음색, 음높이, 발화 속도)과 시간적 패턴(발화 간격, 중첩 여부)을 기반으로 판단됩니다. 모델은 딥러닝 기반 음성 임베딩을 사용하여 각 화자의 고유한 음성 특징을 학습하고, 클러스터링 알고리즘으로 화자를 구분합니다.

3.6.2.2 단계 검수 시스템

- 1단계 STT 결과는 SPEAKER_00/01 라벨로 구분되지만, 부모와 자녀를 정확히 구분하지 못할 수 있습니다. 이를 해결하기 위해 2단계 GPT-5-mini 검수 모델을 도입했습니다.
- 검수 모델은 문맥 기반 분석을 수행합니다. 발화 내용 분석을 통해 명령형/질문형 문장은 부모로, 짧고 단답형 문장은 자녀로 판단합니다. 친밀도 표현을 분석하여 존댓말/격식체는 부모로, 반말/비격식체는 자녀로 구분합니다. 발화 길이를 고려하여 부모의 발화가 일반적으로 더 길다는 패턴을 활용합니다. 논리적 흐름을 파악하여 질문-답변 구조에서 선행 발화는 부모, 후속 응답은 자녀로 추론합니다.
- 검수 프롬프트는 역할 정의("당신은 검수자입니다"), 입력 데이터(STT 출력 JSON), 출력 규칙(Parent/Child 필드만 허용, 타임스탬프 보존, 대화 내용 수정 금지), 화자 구분 로직(문맥 분석,

불확실한 경우 인접 발화 비교)을 포함합니다. 이를 통해 SPEAKER_00/01을 Parent/Child로 정확하게 재라벨링합니다.

3.6.2.3 구조화된 보고서 생성 AI

- GPT-5-mini를 사용하여 대화 분석 및 피드백 보고서를 생성합니다. 프롬프트 엔지니어링 전략으로 JSON 스키마를 엄격하게 정의하여 reportId, subTitle, day, conversationSummary, overallFeedback, expression, changeProposal, emotion, kidAttitude, frequency, strength 필드를 명시합니다.
- 역할 정의("부모-아이 대화 코칭 보고서 생성 비서")를 통해 AI의 전문성을 설정하고, 필수 제약 조건을 부여합니다. STT 오류 가능성을 고려하여 고유명사 언급을 금지하고 대화 방식에 집중하도록 지시합니다. 사용자 프로필(양육 스타일, 자녀 특성, 선호 톤)을 반영하여 맞춤형 피드백을 생성하고, 순수 JSON만 출력하도록 엄격하게 제한하여 파싱 오류를 방지합니다. 한국어로 자연스럽게 작성하고, changeProposal은 정확히 2개 항목, momentEmotion은 3글자 이내로 제한하며, parentConditions/kidConditions에 조건적 발화 구조를 요약하도록 합니다.
- 출력 형식 제어를 위해 시스템 메시지("최종 출력은 반드시 유효한 JSON만 출력하라")를 사용하여 마크다운 코드 블록이나 설명 텍스트를 배제합니다. textwrap.dedent()로 프롬프트 내 불필요한 공백을 제거하여 토큰 사용량을 최적화합니다.

3.6.2.4 다층 분석 알고리즘

- 표현 분석(Expression) 엔진은 부모와 자녀의 언어 사용 패턴을 분석합니다. parentExpression은 질문 유형(개방형/폐쇄형), 명령/제안 비율, 공감 표현 빈도를 분석하고, kidExpression은 문장 복잡도, 주도적/반응적 발화 비율, 감정 표현 다양성을 분석합니다. parentConditions/kidConditions는 조건문("만약~하면"), 선택지 제시("~할래, ~할래?"), 결과 연결("그래서~", "그러니까~") 패턴을 추출합니다.
- 감정 타임라인(Emotion Timeline) 생성은 시계열 감정 변화를 추적합니다. 타임스탬프 기반 세그먼트 분할(15초 단위), 각 구간의 발화 내용에서 감정 키워드 추출(기쁨, 불안, 화남, 만족 등), 3글자 이내 감정 라벨 생성, emotionFeedback으로 전체 감정 흐름 요약을 수행합니다.
- 대화 빈도 분석(Frequency)은 parentFrequency와 kidFrequency를 카운트하여 발화 횟수를 정량화하고, 빈도 비율로 대화 주도권을 분석하며, frequencyFeedback으로 균형 평가 및 개선 방향을 제시합니다. 이상적인 비율은 상황에 따라 다르지만, 아이의 발화 기회가 충분한지를 중점적으로 평가합니다.
- 표현 개선 제안(ChangeProposal)은 정확히 2개의 항목을 생성합니다. existingExpression에서 개선이 필요한 실제 발화를 선택하고, proposalExpression에서 더 효과적인 대안 표현을 제시합니다. 선택 기준은 폐쇄형 질문을 개방형으로 전환, 명령형을 제안형으로 변환, 단답형을 구체적 표현으로 확장하는 것입니다.

3.6.2.5 개인화 및 맥락 인식

- 사용자 프로필 기반 맞춤화는 8개 필드(user_id, child_age, parenting_style, parenting_goal, child_traits, preferred_tone, language, health_issues)를 프롬프트에 주입합니다. child_age에 따라 발달 단계별 적절한 대화 수준을 평가하고, parenting_style에 맞춰 피드백 톤을 조정하며(민주적/권위적/허용적), parenting_goal을 고려하여 목표 지향적 개선 방향을 제시합니다. child_traits를 반영하여 아이의 성향에 맞는 대화 전략을 권장하고, preferred_tone에 따라 부드러운/직설적 피드백 스타일을 적용하며, health_issues가 있으면 특별한 주의사항을 고려합니다.
- STT 오류 대응 전략으로 고유명사 회피(이름, 장소명 언급 금지), 대화 구조 중심 분석(내용보다 패턴에 집중), 문맥 기반 추론(불명확한 단어는 전체 맥락에서 해석)을 적용합니다.

3.6.2.6 JSON 파싱 및 검증 AI

- GPT 출력의 불확실성을 해결하기 위해 다단계 파싱 전략을 사용합니다. 1단계에서 마크다운 코드 블록 제거(``json 태그 제거), 2단계에서 표준 json.loads() 시도, 3단계에서 정규식 {...} 패턴 추출 및 파싱, 4단계에서 정규식 [...] 패턴 추출 후 {"segments": [...]}로 래핑, 최종적으로 모두 실패 시 ValueError 예외 발생을 수행합니다.
- Pydantic 모델 검증을 통해 파싱된 JSON을 VoiceReport 모델로 검증하여 필수 필드 존재 여부, 데이터 타입 일치 여부(reportId는 정수, timeline은 리스트 등), 중첩 구조 정확성(expression, emotion, frequency 객체)을 확인합니다. 검증 실패 시 PARSE_ERROR(500)를 반환하고, 성공 시 타입 안전성이 보장된 객체를 반환합니다.

3.6.2.7 AI 시스템의 강건성 설계

- Fallback 메커니즘으로 STT 실패 시 에러 메시지를 명확히 전달하고, 검수 모델 실패 시 원본 STT 결과를 사용하며, JSON 파싱 실패 시 여러 전략을 순차 시도하고, 최종 실패 시 구조화된 에러 응답을 제공합니다.
- 에러 메시지 개인화를 통해 AUDIO_TOO_LONG은 실제 오디오 길이(duration_sec)를 details에 포함하고, USER_NOT_FOUND는 요청한 user_id를 로그에 기록하며, PARSE_ERROR는 GPT 원본 출력을 details에 포함하여 디버깅을 지원합니다.
- 로깅 및 모니터링을 위해 생성된 프롬프트를 서버 로그에 기록하고, GPT 응답 시간을 측정하며(향후 개선), 에러 발생 빈도를 추적하고(향후 개선), 사용자별 요청 패턴을 분석합니다(향후 개선).

3.6.2.8 AI 윤리 및 프라이버시

- 데이터 프라이버시 보호를 위해 오디오 파일은 처리 후 즉시 삭제(finally 블록)하고, 대화 내용은 보고서 생성 후 메모리에서 제거하며, RDS에는 사용자 프로파일만 저장하고 대화 기록은 저장하지 않습니다. UUID 기반 파일명으로 충돌을 방지합니다.
- 투명성 확보를 위해 AI 생성 보고서임을 명시하고(향후 UI 개선), 피드백이 제안일 뿐 절대적 기준이 아님을 강조하며, 전문가 상담이 필요한 경우 명시적으로 권장합니다.

Appendix) 사용된 오픈소스 / 외부 모듈

URL	이름	용도
https://fastapi.tiangolo.com/	FastAPI	공통: API 서버
https://docs.pydantic.dev/latest/	Pydantic	공통: 요청/응답 모델
https://platform.openai.com/docs/	OpenAI API	공통: GPT 호출
https://pandas.pydata.org/	Pandas	공통: 데이터 처리
https://docs.python.org/3/library/pathlib.html	Pathlib, JSON, OS	공통:파일/시스템 유틸리티
https://sbert.net/	SentenceTransformers	챗봇: 텍스트 데이터 임베딩
https://faiss.ai/	FAISS	챗봇: 인덱스 생성
https://github.com/openai/whisper	Whisper	보이스리포트: STT
https://pyannote.github.io/pyannote-audio/	Pyannote	보이스리포트: 화자분리

https://platform.openai.com/docs/guides/speech-to-text#improving-reliability	gpt-4o-transcribe-diarize	보이스리포트: STT + 화자 분리 통합
https://docs.spring.io/spring-boot/reference/web/index.html#page-title	spring-boot-starter-web	웹: REST API
https://docs.spring.io/spring-boot/reference/using/build-systems.html#using.build-systems.starters	spring-boot-starter-validation	검증: 요청 DTO 검증
https://docs.spring.io/spring-boot/reference/data/sql.html#data.sql.jpa-and-spring-data	spring-boot-starter-data-jpa	데이터베이스: ORM/JPA
https://docs.spring.io/spring-boot/reference/web/spring-security.html#page-title	spring-security-crypto	보안: BCrypt 암호화
https://projectlombok.org/features/	lombok	개발 편의: 보일러플레이트 코드 제거
https://docs.spring.io/spring-boot/reference/testing/index.html#page-title	spring-boot-starter-test	테스트: 기본 테스트 라이브러리